

Capitolo 40 — PROT-011 — Capability Evidence Check Before Block

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-40
Data master	2026-08-31
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/40_prot_011_capability_evidence_check_before_block.md
Source SHA	b45eddb
Release	2026-09-04T09:45:54.479Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-31 **Scope:** WCM generale, domain-agnostic

40.0 Prima di dire «non posso», bisogna sapere se è vero

In qualunque organizzazione può accadere che un'attività sembri impossibile semplicemente perché, in quel momento, non è evidente con quali strumenti possa essere svolta.

Una persona apre un cassetto, non trova subito ciò che cerca e conclude che quell'oggetto non esiste. Un'altra controlla l'inventario, verifica il magazzino, chiede se esiste un'alternativa autorizzata e solo dopo stabilisce che l'oggetto manca davvero.

La differenza non è marginale. Nel primo caso una supposizione diventa una decisione operativa. Nel secondo, la decisione nasce da un'evidenza.

PROT-011 — Capability Evidence Check Before Block applica lo stesso principio al WCM.

Una **capability** è, in termini semplici, la possibilità tecnica di compiere una certa azione nel runtime corrente: leggere una fonte, usare un connettore, inviare un messaggio, accedere a un servizio, eseguire una funzione prevista.

Il protocollo impedisce che l'assenza di questa capacità venga dichiarata per memoria, abitudine o impressione.

La sua regola centrale è semplice:

Prima di concludere che una capacità non esiste, occorre verificarlo nel contesto corrente quando esiste un meccanismo di verifica applicabile.

Non significa provare tutto ogni volta. Significa non trasformare un'impressione negativa in un fatto organizzativo.

40.1 Il problema che PROT-011 risolve

Un sistema che opera attraverso strumenti, connector e servizi può cambiare da una run all'altra.

Una capability che ieri non era disponibile potrebbe esserlo oggi. Un connettore potrebbe essere presente ma non ancora individuato. Un servizio potrebbe esistere come fallback autorizzato. Oppure lo strumento potrebbe essere disponibile ma temporaneamente inutilizzabile per un problema di autenticazione, permessi, rate limit o indisponibilità tecnica.

Senza una regola esplicita, situazioni molto diverse rischiano di essere ridotte alla stessa frase:

| «Non posso farlo.»

Quella frase può nascondere almeno quattro condizioni differenti:

```
LA CAPABILITY ESISTE ED È UTILIZZABILE
LA CAPABILITY ESISTE MA È TEMPORANEAMENTE BLOCCATA
LA CAPABILITY NON È VERIFICABILE CON LE EVIDENZE DISPONIBILI
LA CAPABILITY MANCA DAVVERO
```

Confonderle produce effetti concreti. Un workflow può fermarsi quando avrebbe potuto continuare. Un lavoro può essere delegato inutilmente. Un blocco temporaneo può essere registrato come limite strutturale. Una supposizione può propagarsi alle run successive come se fosse conoscenza certa.

PROT-011 nasce per impedire questo salto logico.

40.2 Capability non significa authority

Prima di entrare nel flusso operativo è necessario separare due concetti.

Sapere **come** fare qualcosa non significa essere autorizzati a farlo.

Il protocollo verifica la disponibilità tecnica di una capability. Non concede permessi, non supera gate, non modifica scope e non sostituisce le regole di governance.

La relazione può essere espressa così:

```
CAPABILITY AVAILABLE
≠
ACTION AUTHORIZED
```

Un esempio quotidiano aiuta. Avere fisicamente la chiave di una stanza dimostra una possibilità tecnica di accesso; non dimostra automaticamente il diritto organizzativo di entrare in quella stanza.

PROT-011 risponde alla domanda:

| «La capacità tecnica esiste davvero?»

Le regole di authority rispondono invece alla domanda:

| «Anche se esiste, questa azione è autorizzata?»

Questa separazione è essenziale perché il protocollo non diventi una scorciatoia per aggirare la governance.

40.3 Il trigger

PROT-011 non si attiva prima di ogni singola azione.

Si attiva quando una decisione operativa importante potrebbe dipendere da una **conclusione negativa sulla disponibilità di una capability**.

Per esempio, il protocollo diventa rilevante prima di:

- dichiarare un **CAPABILITY_GAP**;
- dire che una capability necessaria non è disponibile;
- affermare che un connector, un tool o un service non può essere usato;
- interrompere un workflow perché si presume che manchi la capacità tecnica necessaria;
- cambiare routing sulla base di quella presunta assenza.

Se invece la capability è già stata accertata nella stessa run con evidenza sufficiente, non è richiesta una nuova discovery solo per ripetere ciò che è già noto.

Il protocollo non impone quindi verifica continua. Impone **verifica prima della negazione**.

40.4 Gli input necessari

La Capability Evidence Check parte da una domanda concreta, non da una ricerca generica di tutti gli strumenti esistenti.

Gli input minimi sono:

- l'azione che si vuole eseguire;
- la capability tecnica realmente necessaria per quell'azione;
- ciò che è già stato verificato nella run corrente;
- i meccanismi disponibili per discovery o verifica;
- l'eventuale presenza di un service autorizzato che possa fornire la capability quando quella diretta manca;
- il contesto di authority applicabile all'azione finale.

La prima domanda è quindi molto semplice:

| Di quale capacità concreta ho bisogno?

Senza questa precisione, la verifica rischia di diventare un'esplorazione indiscriminata e inefficiente.

40.5 Il flusso minimo

Il percorso canonico parte dalla capability diretta.

AZIONE RICHIEDE UNA CAPABILITY

↓

```
CAPABILITY DIRETTA GIÀ ACCERTATA NELLA RUN?
├─ SÌ → nessuna nuova discovery necessaria
└─ NO / INCERTA
    ↓
CAPABILITY EVIDENCE CHECK REALE
    ↓
DIRECT DISPONIBILE?
├─ SÌ → routing DIRECT secondo PROT-003
└─ NO
    ↓
SERVICE AUTORIZZATO DISPONIBILE?
├─ SÌ → routing SERVICE_REQUIRED secondo PROT-003
└─ NO → valutazione CAPABILITY_GAP
```

La sequenza contiene una disciplina importante: **il gap strutturale viene valutato alla fine, non all'inizio.**

Prima si verifica la capacità diretta. Se manca, si verifica l'eventuale fallback autorizzato. Solo quando entrambe le strade risultano realmente assenti può emergere un **CAPABILITY_GAP** nel significato già definito dal WCM.

40.6 Che cosa conta come evidenza

Non tutte le verifiche devono essere identiche.

PROT-011 richiede il meccanismo più diretto e proporzionato alla decisione da prendere.

A seconda del caso, l'evidenza può derivare da:

- discovery delle funzioni esposte da un connector;
- lettura della superficie di capability resa disponibile dal runtime;
- verifica di connessione o autenticazione quando serve a stabilire l'utilizzabilità;
- tentativo tecnico non distruttivo previsto dallo strumento;
- verifica dell'esistenza di un service autorizzato pertinente.

Il punto non è eseguire per forza l'azione finale.

Se una discovery affidabile dimostra già che la capability esiste, non serve compiere un'operazione reale solo per provarne l'esistenza.

L'evidenza deve essere sufficiente **per la classificazione che stiamo per fare.**

40.7 Gate 1 — La capability è già stata accertata nella run?

Il primo gate evita l'eccesso opposto: verificare continuamente la stessa cosa.

Se nella run corrente esiste già evidenza affidabile che la capability è disponibile e utilizzabile entro i limiti rilevanti, PROT-011 non richiede di ripetere la discovery.

Questo protegge efficienza e continuità.

La regola non è quindi:

| *«Verifica sempre tutto.»*

È:

«Non dichiarare assenza senza una verifica contemporanea o un'evidenza già valida nella run.»

40.8 Gate 2 — Disponibile direttamente o no?

Quando la capability non è già accertata, viene eseguita una verifica reale.

Se il risultato dimostra che la capability diretta esiste ed è utilizzabile, la classificazione è:

CAPABILITY_AVAILABLE

Da quel momento entra in gioco **PROT-003 – Direct Before Delegate**: se l'azione è anche autorizzata e il routing è **DIRECT**, viene usata la capacità diretta.

PROT-011 non decide autonomamente l'intero routing. Fornisce a PROT-003 un fatto più affidabile: la capability diretta è stata verificata, invece di essere supposta presente o assente.

40.9 Gate 3 — Mancanza reale o blocco temporaneo?

Una capability può esistere senza essere utilizzabile in quel momento.

Questa distinzione è uno dei punti più importanti del protocollo.

Se un connector è presente ma l'autenticazione è scaduta, il problema non è che la capability non esista. Lo stesso vale per un rate limit, un outage, un permesso insufficiente o un errore tecnico contingente.

In questi casi la classificazione è:

TEMPORARY_CAPABILITY_BLOCK

Un blocco temporaneo non deve essere promosso a **CAPABILITY_GAP**.

Se impedisce materialmente di proseguire, la run può essere classificata **INTERRUPTED / RESUMABLE** quando applicabile, registrando causa e next action.

L'organizzazione conserva così una memoria più precisa:

NON POSSO ORA
≠
NON ESISTE IL MODO DI FARLO

40.10 Gate 4 — E se non posso nemmeno verificarlo?

Esiste una terza possibilità: il runtime non offre evidenza sufficiente per stabilire se la capability sia presente oppure no.

La classificazione corretta è allora:

CAPABILITY_UNVERIFIED

Questa categoria protegge il sistema da un errore epistemico molto comune: trasformare l'incertezza in certezza negativa.

Se non possiamo dimostrare che una capacità esista, non possiamo automaticamente concludere che non esista.

Se la capability è necessaria per continuare, il workflow applicherà la stop o l'escalation coerente con il proprio contratto, ma la causa resterà correttamente descritta come **non verificata**, non come gap strutturale.

40.11 Gate 5 — Esiste un service autorizzato?

Quando la capability diretta risulta realmente assente, il protocollo non conclude ancora automaticamente **CAPABILITY_GAP**.

Prima deve essere verificata, quando pertinente, l'esistenza di un service validato e autorizzato capace di fornire quella funzione nel perimetro corrente.

Se esiste, il routing può diventare:

SERVICE_REQUIRED

secondo PROT-003 e con la delega minima necessaria.

Solo quando:

DIRECT NON DISPONIBILE
+
NESSUN SERVICE VALIDATO/AUTORIZZATO DISPONIBILE

può essere classificato:

CAPABILITY_GAP

Il protocollo non autorizza a inventare un connector, un accesso o un service inesistente per evitare il gap. Verificare prima di negare non significa fingere una capacità che non c'è.

40.12 Le quattro classificazioni da non confondere

La Capability Evidence Check produce quindi una distinzione più precisa rispetto al semplice sì/no.

CAPABILITY_AVAILABLE

La capacità tecnica esiste ed è utilizzabile nel runtime corrente, entro i limiti rilevati.

TEMPORARY_CAPABILITY_BLOCK

La capacità esiste, ma un ostacolo contingente ne impedisce l'uso nella run corrente.

CAPABILITY_UNVERIFIED

Le evidenze disponibili non permettono di stabilire in modo affidabile se la capacità esista o sia utilizzabile.

CAPABILITY_GAP

La capacità diretta manca realmente e non esiste un service validato/autorizzato che possa fornirla nel perimetro corrente.

Questa tassonomia evita che quattro realtà operative diverse vengano compresse nella stessa conclusione.

40.13 L'output: una conclusione ricostruibile

Quando una run si ferma o cambia routing per una conclusione negativa di capability, il risultato deve poter essere ricostruito.

Non è richiesto un unico formato persistente universale se il workflow non lo prevede. È richiesta però la sostanza dell'evidenza.

Il reporting deve rendere comprensibili almeno questi elementi:

```
CAPABILITY richiesta  
CHECK eseguita  
DIRECT result  
SERVICE fallback result  
CLASSIFICAZIONE finale  
NEXT ACTION
```

Questo trasforma una frase generica come «strumento non disponibile» in una conclusione verificabile.

Chi riprende il lavoro può capire cosa è stato controllato, che cosa è risultato disponibile, che cosa è mancato e quale sia il passo successivo.

40.14 Relazione con PROT-003 — Direct Before Delegate

PROT-011 e PROT-003 rispondono a due domande consecutive.

PROT-011 chiede:

| Che cosa è realmente disponibile?

PROT-003 chiede:

| Dato ciò che è disponibile, qual è il routing minimo corretto?

La relazione è quindi:

```
PROT-011
VERIFICA LA REALTÀ DELLA CAPABILITY
↓
PROT-003
ROUTA DIRECT / LOCAL_REQUIRED / SERVICE_REQUIRED / CAPABILITY_GAP
```

PROT-011 è il prerequisito epistemico di PROT-003 quando il routing dipende da una risposta negativa sulla capability diretta.

Il termine *epistemico* qui significa semplicemente: riguarda ciò che il sistema può considerare realmente conosciuto e dimostrato.

40.15 Relazione con PROT-009 — Contiguous Workflow Execution

PROT-009 richiede che un workflow prosegua fino a una vera stop condition.

Una capability mancante può essere una stop condition reale, ma solo se la sua assenza è stata verificata.

Senza PROT-011, una falsa percezione di impossibilità potrebbe interrompere prematuramente un workflow contiguo.

La relazione è quindi molto concreta:

```
PRESUNTO BLOCCO DI CAPABILITY
↓
PROT-011 VERIFICA
↓
BLOCCO REALE?
├ NO → il workflow continua secondo il routing applicabile
└ SÌ → si applica la stop condition corretta
```

Se il problema è temporaneo, la semantica corretta è normalmente quella di una interruzione riprendibile, non quella di un completamento e non quella di un gap strutturale.

40.16 Failure mode

I failure mode principali non riguardano soltanto strumenti che non funzionano. Riguardano soprattutto classificazioni sbagliate.

Memoria scambiata per evidenza

Una capability assente in una run precedente viene considerata assente anche oggi senza nuova verifica.

Effetto: falsa propagazione del gap.

Mancata discovery

Lo strumento non è immediatamente visibile e l'assenza viene dichiarata senza cercare il meccanismo previsto per scoprirlo.

Effetto: falso `CAPABILITY_GAP`.

Blocco temporaneo scambiato per assenza

Il connector esiste ma l'autenticazione fallisce, c'è rate limit o un outage.

Effetto: un problema contingente viene registrato come limite strutturale.

Incertezza trasformata in negazione

Le evidenze non bastano, ma il sistema conclude comunque che la capability non esista.

Effetto: `CAPABILITY_UNVERIFIED` viene erroneamente trasformato in `CAPABILITY_GAP`.

Fallback non verificato

La capability diretta manca e il gap viene dichiarato prima di controllare l'eventuale service autorizzato pertinente.

Effetto: interruzione evitabile o routing incompleto.

Capability trasformata in authority

La capability viene trovata e il sistema presume che questo basti per poter agire.

Effetto: violazione della governance.

40.17 Il gate rapido prima di bloccare

Prima di una conclusione negativa, il protocollo richiede che siano ricostruibili cinque risposte:

1. **Quale capability concreta serve?**
2. **Come è stata verificata nella run corrente?**
3. **Il risultato è assenza, blocco temporaneo o incertezza?**
4. **Se Direct manca, è stato verificato il service autorizzato pertinente?**
5. **La classificazione finale è coerente con PROT-003?**

Se queste risposte non sono disponibili, il `CAPABILITY_GAP` non è ancora dimostrato.

Questo gate non pretende infallibilità. Pretende che una conclusione negativa sia fondata su ciò che è stato realmente osservato.

40.18 Un esempio astratto

Immaginiamo un'organizzazione che debba consegnare un documento attraverso un certo canale.

Il primo operatore non vede immediatamente il meccanismo di consegna e conclude:

| «*Quel canale non è disponibile.*»

PROT-011 impedisce di fermarsi lì.

Il percorso corretto è:

```
Serve la capability di consegna
→ non è ancora accertata nella run
→ viene eseguita la discovery prevista
→ il meccanismo esiste
→ la capability è CAPABILITY_AVAILABLE
→ solo dopo si verifica se l'azione è autorizzata
```

In una variante diversa, il meccanismo esiste ma l'accesso è temporaneamente scaduto:

```
capability presente
→ autenticazione non valida
→ TEMPORARY_CAPABILITY_BLOCK
→ non CAPABILITY_GAP
```

In una terza variante, né capability diretta né service autorizzato esistono:

```
DIRECT assente verificato
→ SERVICE assente verificato
→ CAPABILITY_GAP
```

Le tre situazioni possono sembrare simili all'inizio. Solo la verifica permette di distinguerle correttamente.

40.19 Maturity e limiti

La baseline corrente di **PROT-011** ha stato:

```
VALIDATED BY GOVERNANCE / FIELD VALIDATION IN PROGRESS
```

Questo significa che il protocollo è stato accettato come regola di governance WCM, ma la sua validazione sul campo è ancora in corso.

Non implica che ogni possibile runtime, connector, service o failure tecnica sia già stato osservato e dimostrato in ogni contesto.

Il protocollo ha inoltre limiti precisi:

- non garantisce che ogni runtime renda sempre verificabile ogni capability;
- non elimina blocchi tecnici reali;
- non crea strumenti mancanti;
- non conferisce authority;
- non richiede discovery ripetitiva quando l'evidenza è già valida nella stessa run;
- non sostituisce PROT-003 nel routing;
- non inventa uno schema persistente universale di reporting quando il workflow non lo prevede.

Il suo compito è più ristretto e fondamentale: migliorare la qualità della conclusione prima che una presunta impossibilità diventi una decisione operativa.

40.20 Source map

La base tecnica del capitolo è:

- [wcm/process-book/protocols/PROT-011_CAPABILITY_EVIDENCE_CHECK_BEFORE_BLOCK.md](#) — fonte canonica primaria;
- [wcm/documentation/process-memory-book/BOOK_INDEX.md](#) — mapping editoriale CH40 ↔ PROT-011;
- [PROT-003 – Direct Before Delegate](#) — relazione canonica per il routing successivo alla verifica;
- [PROT-009 – Contiguous Workflow Execution](#) — relazione canonica per la validità delle stop condition e la continuità del workflow.

Il capitolo non modifica la semantica di questi elementi e non introduce nuove classi, gate o authority rispetto alla baseline corrente.

40.21 La regola da ricordare

Un sistema affidabile non deve confondere ciò che non vede subito con ciò che non esiste.

La regola finale di PROT-011 è:

Una conclusione negativa sulla capability richiede evidenza corrente: prima di dire “non posso”, verifica se davvero non puoi.

E subito dopo resta valida una seconda separazione:

Sapere che puoi farlo non significa ancora che sei autorizzato a farlo.